Sorting and Removing Duplicates from a Linked List

Kevin Browne

How can we sort a linked list?

Sorting a linked list

- We can apply sorting algorithms to linked lists ***more or less the same*** as arrays in many cases
  - Only instead if incrementing array counters, we traverse the list
  - Instead of swapping array values, we swap node values

- Bubble sort translates nicely to working with a linked list… though we may want to implement a faster algorithm!
  - https://en.wikipedia.org/wiki/Bubble_sort

- Empty lists and lists with one node are already "sorted"

Sorting a list

- While it's technically possible to re-arrange the nodes themselves by changing pointers, does this even always make sense?
  - The list **length** is not going to change after all
  - So why not just swap the values in the nodes? Especially if the value is something short like an int value.

- If swap involves copying a large struct's worth of values (e.g. a struct containing a large array), perhaps we ***do*** need to re-arrange nodes via changing pointers instead!

Key bubble sort implementation with linked list

```
bool swapped = false;

do
{
  swapped = false;
  Node *current = head;
  Node *prev = NULL;

  while (current->next != NULL)
  {
    prev = current;
    current = current->next;

    if (current != NULL)
    {
      if (current->value < prev->value)
      {
        int temp;
        temp = prev->value;
        prev->value = current->value;
        current->value = temp;
        swapped = true;
      }
    }
  }
} while (swapped);
```

Testing the sort list function

```
// Test the sort_list function
Node *list3 = NULL;
list3 = insert_at_head(list3, 5);
list3 = insert_at_head(list3, 9);
list3 = insert_at_head(list3, 1);
list3 = insert_at_head(list3, 8);
list3 = insert_at_head(list3, 4);
list3 = insert_at_head(list3, 7);
list3 = insert_at_head(list3, 3);
list3 = insert_at_head(list3, 6);
list3 = insert_at_head(list3, 2);
printf("\nList 3 before sort:\n");
print_list(list3);
sort_list(list3);
printf("\nList 3 after sort:\n");
print_list(list3);
```

Sort list function test results...

```
List 3 before sort:
Node 0: 2
Node 1: 6
Node 2: 3
Node 3: 7
Node 4: 4
Node 5: 8
Node 6: 1
Node 7: 9
Node 8: 5
```

Sort list function test results...

```
List 3 after sort:
Node 0: 1
Node 1: 2
Node 2: 3
Node 3: 4
Node 4: 5
Node 5: 6
Node 6: 7
Node 7: 8
Node 8: 9
```

How can remove duplicate elements from a list?

Deleting duplicates

- To delete duplicates we can use two pointers to nodes to traverse the list
  - An **outer loop** can keep track of the current node
  - An **inner loop** can delete any duplicates of the current node from the list

- The inner loop can consider the value of the outer loop's current node, and traverse the list with its own "current node"
  - If the value of a node is equal to the value being considered, we can remove the node from the list, and keep traversing the list
  - If the value of a node is not equal to the value being considered, we can just keep traversing the list

Delete duplicates function

```
void delete_duplicates(Node *head)
{
  if (head == NULL) return ;
  if (head->next == NULL) return ;

  Node *current1, *current2, *duplicate;
  current1 = head;

  while (current1 != NULL && current1->next != NULL)
  {
    current2 = current1;
    while (current2->next != NULL)
    {
      if (current1->value == current2->next->value)
      {
        duplicate = current2->next;
        current2->next = current2->next->next;
        free(duplicate);
      }
      else current2 = current2->next;
    }
    current1 = current1->next;
  }
}
```

Testing delete duplicates function

```
// Test the delete_duplicates function
Node *list4 = NULL;
list4 = insert_at_head(list4, 5);
list4 = insert_at_head(list4, 5);
list4 = insert_at_head(list4, 6);
list4 = insert_at_head(list4, 7);
list4 = insert_at_head(list4, 8);
list4 = insert_at_head(list4, 1);
list4 = insert_at_head(list4, 6);
list4 = insert_at_head(list4, 8);
list4 = insert_at_head(list4, 2);
list4 = insert_at_head(list4, 8);
list4 = insert_at_head(list4, 9);
list4 = insert_at_head(list4, 7);
list4 = insert_at_head(list4, 3);
list4 = insert_at_head(list4, 6);
printf("\nList 4 before delete duplicates:\n");
print_list(list4);
delete_duplicates(list4);
printf("\nList 4 after delete duplicates:\n");
print_list(list4);
```

Delete duplicates test results…

```
List 4 before delete duplicates:
Node 0: 6
Node 1: 3
Node 2: 7
Node 3: 9
Node 4: 8
Node 5: 2
Node 6: 8
Node 7: 6
Node 8: 1
Node 9: 8
Node 10: 7
Node 11: 6
Node 12: 5
Node 13: 5
```

Delete duplicates test results…

```
List 4 after delete duplicates:
Node 0: 6
Node 1: 3
Node 2: 7
Node 3: 9
Node 4: 8
Node 5: 2
Node 6: 1
Node 7: 5
```